

Relatório Técnico do Trabalho Prático 3 — Algoritmos de Caminhos Mínimos em Grafos

Isabela M. Andrade, Júlia A. da Rocha, Otávio de Oliveira, Pedro Brassi Luccas
Departamento de Ciências da Computação — Universidade Federal de Alfenas
Avenida Jovino Fernandes de Sales 2600 — CEP 37133-840 — Alfenas, MG, Brasil
{isabela.mageste, juliaalves.rocha, otavio.oliveira1, pedro.luccas1}@sou.unifal-mg.edu.br

Resumo. Este trabalho aborda o problema de caminhos mínimos de fonte única (SSSP) em grafos ponderados, conexos e não orientados. Foram implementados e avaliados três algoritmos distintos: o algoritmo de Dijkstra, o algoritmo de Bellman-Ford e uma implementação inspirada no algoritmo de Duan (2025). O objetivo principal foi comparar empiricamente o comportamento dos algoritmos em relação ao tempo de execução, complexidade computacional e escalabilidade. Foram executados 30 testes sobre três topologias de grafos aleatórios (Barabási-Albert, Erdős-Rényi e Watts-Strogatz) em cinco tamanhos ($n = 500, 1000, 2000, 5000$ e 10000). Os resultados confirmam que Dijkstra apresenta crescimento supralinear próximo do quadrático, Bellman-Ford é consistentemente mais lento que os demais, e a implementação inspirada em Duan apresenta tempos competitivos com Dijkstra (chegando a ser $\sim 2\times$ mais rápida em grafos Barabási e Watts com 10 000 vértices). Adicionalmente, observou-se uma divergência sistemática no custo retornado pela implementação atual do Duan em relação à solução ótima, indicando a necessidade de refinamento da rotina.

1. Introdução

O problema de caminho mais curto (Single-Source Shortest Path, SSSP) consiste em determinar a menor distância entre um vértice de origem e os demais vértices de um grafo ponderado. Esse problema é um dos mais importantes da teoria dos grafos e possui aplicações em diversas áreas, como sistemas de navegação e GPS, redes de computadores, transporte urbano, jogos digitais, redes logísticas e inteligência artificial.

No contexto deste trabalho, foram analisados algoritmos de caminho mínimo para grafos ponderados, conexos e não orientados. O objetivo principal é comparar diferentes paradigmas de solução, observando suas diferenças em relação ao custo computacional, eficiência prática e crescimento assintótico.

O trabalho foi desenvolvido utilizando três algoritmos distintos:

- Algoritmo de Dijkstra;
- Algoritmo de Bellman-Ford;
- Algoritmo de Duan.

O algoritmo de Dijkstra representa a abordagem gulosa clássica para caminhos mínimos. Já o Bellman-Ford utiliza relaxamentos sucessivos de todas as arestas do grafo, enquanto o

algoritmo de Duan propõe uma estratégia híbrida baseada em relaxamentos locais, redução de fronteira e refinamento hierárquico do processamento.

2. Metodologia

O projeto foi desenvolvido em linguagem C, utilizando o código-base disponibilizado pelo professor. A implementação foi organizada nos arquivos algoritmos.c e algoritmos.h. O arquivo base.c, fornecido originalmente, foi mantido como estrutura principal de leitura dos grafos, medição de tempo e execução dos algoritmos. Os grafos foram representados utilizando matrizes de adjacência alocadas dinamicamente.

2.1. Representação do grafo

O grafo é representado pela seguinte estrutura:

```
typedef struct {
    int num_nos;
    int num_arestas;
    int **matriz;
} Grafo;
```

Cada posição `matriz[u][v]` armazena o peso da aresta entre os vértices `u` e `v`. Quando não existe aresta entre dois vértices, o valor armazenado é 0. Os grafos utilizados possuem as seguintes características: ponderados, conexos e não orientados.

2.2. Algoritmo de Dijkstra

O algoritmo de Dijkstra calcula os menores caminhos a partir de um vértice de origem até todos os demais vértices do grafo. A implementação utiliza matriz de adjacência, busca linear do menor vértice, um vetor de distâncias e um vetor de vértices visitados.

2.2.1. Inicialização

Inicialmente, são criados dois vetores auxiliares:

```
double *dist;
bool *visitado;
```

O vetor `dist` armazena as menores distâncias conhecidas até cada vértice, enquanto `visitado` controla os vértices cuja menor distância já foi definitivamente determinada. Todas as distâncias são inicializadas com `INT_MAX / 2` (representando infinito) e a origem recebe distância zero. O uso da divisão por dois evita problemas de overflow durante as somas realizadas no relaxamento das arestas.

2.2.2. Pseudocódigo

```
Algoritmo Dijkstra(matriz, n, origem)
    para i ← 0 até n-1 faça
        dist[i] ← INFINITO
        visitado[i] ← falso
    fim-para
    dist[origem] ← 0
    para iter ← 0 até n-1 faça
```



```

        mudou ← verdadeiro
    fim-se
    fim-se
    fim-para
    fim-para
    se mudou = falso então
        pare      // parada antecipada
    fim-se
    fim-para
fim

```

2.3.4. Otimização utilizada

Foi adicionada uma otimização de parada antecipada: caso uma iteração completa do laço externo não altere nenhuma distância, o algoritmo é encerrado, evitando iterações desnecessárias.

2.3.5. Complexidade

Devido ao uso de matriz de adjacência e três laços aninhados, a complexidade no pior caso é $O(V^3)$. Para grafos esparsos representados por lista de arestas, a complexidade reduz-se para $O(V \cdot E)$, sendo essa a notação adotada na análise comparativa deste trabalho (Tabela 1).

2.4. Algoritmo de Duan

O algoritmo de Duan et al. (2025) representa um avanço relevante na área de algoritmos de grafos, sendo o primeiro algoritmo determinístico para SSSP a atingir complexidade sublogarítmica no fator associado à ordenação de vértices, com resultado teórico $O(m \cdot \log^{(2/3)} n)$, superando a barreira clássica de Dijkstra $O(m + n \log n)$.

A principal contribuição do algoritmo consiste na substituição da ordenação global de vértices por uma estrutura hierárquica baseada em redução progressiva de fronteira, recursão controlada e relaxamentos locais em múltiplas escalas. A implementação desenvolvida foi inspirada no modelo apresentado no artigo original.

2.4.1. Parâmetros fundamentais

O algoritmo introduz dois parâmetros fundamentais que controlam o equilíbrio entre os paradigmas de Dijkstra (guloso) e Bellman-Ford (iterativo):

```

k = ⌊log(1/3)(n)⌋
t = ⌊log(2/3)(n)⌋

```

O parâmetro k controla as iterações locais de relaxamento (estilo Bellman-Ford) responsáveis por propagar distâncias para regiões próximas no grafo, sem depender de extrações repetidas de uma fila de prioridade global. O parâmetro t controla o fator de divisão aplicado em cada nível do processo recursivo, organizando o problema em subestruturas menores.

2.4.2. O subproblema central: BMSSP

O funcionamento do algoritmo principal depende de um subprocedimento denominado BMSSP (Bounded Multi-Source Shortest Path), responsável por calcular caminhos mínimos

de maneira limitada dentro de regiões do grafo. O procedimento BMSSP recebe três parâmetros: o nível l da recursão, um limite superior de distância B e um conjunto S de vértices da fronteira. Ao término, retorna um novo limite refinado $B' \leq B$ e o conjunto U dos vértices considerados completos, isto é, cuja distância mínima $d(v)$ já foi corretamente determinada e satisfaz $d(v) < B'$.

2.4.3. FindPivots — redução de fronteira

O procedimento FindPivots constitui o núcleo do BMSSP. Seu objetivo é reduzir o conjunto de fronteira S a um subconjunto significativamente menor de vértices estratégicos, denominados pivôs P . A redução é fundamental para diminuir o custo associado ao processamento dos vértices candidatos. O procedimento executa k iterações de relaxamento local estilo Bellman-Ford. Caso o conjunto W cresça além de um limite proporcional a $k \cdot |S|$, a fronteira é considerada excessivamente densa e o algoritmo retorna $P = S$ imediatamente. Caso contrário, é construída uma floresta de caminhos mínimos induzida por W , e os vértices de S cujas subárvores possuem tamanho suficiente são selecionados como pivôs.

2.4.4. Pseudocódigo de FindPivots

```

Algoritmo FindPivots( $B, S, db, graph, k$ )
   $W \leftarrow S$ 
   $W_{prev} \leftarrow S$ 
  para  $i \leftarrow 1$  até  $k$  faça
     $W_{next} \leftarrow$  vazio
    para cada  $u$  em  $W_{prev}$  faça
      para cada  $(u, v)$  em  $E$  faça
        se  $db[u] + w(u, v) \leq db[v]$  e  $db[u] + w(u, v) < B$  então
           $db[v] \leftarrow db[u] + w(u, v)$ 
           $W_{next} \leftarrow W_{next} \cup \{v\}$ 
        fim-se
      fim-para
    fim-para
     $W \leftarrow W \cup W_{next}$ 
     $W_{prev} \leftarrow W_{next}$ 
    se  $|W| > k \cdot |S|$  então
      retorne  $(S, W)$ 
    fim-se
  fim-para
  Construir floresta  $F$  de caminhos mínimos induzida por  $W$ 
   $P \leftarrow$  vazio
  para cada  $u$  em  $S$  faça
    se tamanho_subarvore( $u, F$ )  $\geq k$  então
       $P \leftarrow P \cup \{u\}$ 
    fim-se
  fim-para
  retorne  $(P, W)$ 
fim

```

2.4.5. Pseudocódigo de BMSSP

```

Algoritmo BMSSP( $l, B, S$ )
  se  $l = 0$  então
    Executar Dijkstra limitado em  $S$ 

```

```

    retorne (B', U)
fim-se
(P, W) ← FindPivots(B, S, db, graph, k)
Inserir P em estrutura auxiliar D
U ← vazio
enquanto houver blocos em D faça
    (Bi, Si) ← D.extract()
    (B'i, Ui) ← BMSSP(l-1, Bi, Si)
    U ← U uniao Ui
    para cada u em Ui faça
        Relaxar arestas de u
    fim-para
fim-enquanto
retorne (B', U)
fim

```

2.4.6. Estrutura da fronteira na implementação

Na implementação desenvolvida, o algoritmo mantém os conjuntos auxiliares S, W, W_{prox} e pivots, que representam, respectivamente: a fronteira atual, os vértices alcançados, os novos vértices descobertos na iteração corrente e os pivôs selecionados. Os vértices da fronteira são ordenados localmente por distância e processados em ordem crescente, simulando um Dijkstra restrito.

2.4.7. Complexidade

A implementação é inspirada na complexidade teórica $O(m \cdot \log^{(2/3)} n)$. Entretanto, devido ao uso de matriz de adjacência e a simplificações práticas adotadas, o comportamento empírico pode diferir da análise assintótica original, conforme discutido na Seção 4.

3. Comparação teórica

A Tabela 1 resume as complexidades e características dos três algoritmos analisados.

Algoritmo	Complexidade	Características
Dijkstra	$O(V^2)$	Abordagem gulosa
Bellman-Ford	$O(V \cdot E)$	Relaxamento global
Duan	$O(m \log^{(2/3)} n)$	Relaxamento local e pivôs

Tabela 1. Resumo das complexidades dos algoritmos analisados.

3.1. Estrutura experimental

Os experimentos foram realizados utilizando grafos aleatórios sintéticos gerados em três topologias distintas, em cinco tamanhos de entrada ($n = 500, 1000, 2000, 5000$ e 10000 vértices). Para cada combinação (topologia $\times$ tamanho), foram geradas duas instâncias com sementes diferentes, totalizando 30 testes. Para cada instância, foram medidos o custo total dos caminhos mínimos (soma das menores distâncias a partir do vértice de origem) e o tempo

de execução. Os tempos foram medidos utilizando a função `clock()` da biblioteca padrão de C.

3.1.1. Topologias utilizadas

Foram empregados três modelos clássicos de geração de grafos aleatórios, com o objetivo de cobrir diferentes regimes de densidade e estrutura:

- Barabási-Albert ($m = 3$): modelo de crescimento por anexação preferencial, que produz redes livres de escala. Apresenta poucos vértices com alto grau (hubs) e muitos vértices com baixo grau; grafo esparso.
- Erdős-Rényi ($p = 0,01$): modelo aleatório uniforme em que cada par de vértices é conectado de forma independente com probabilidade p . Tende a produzir grafos com distribuição de grau aproximadamente binomial e densidade intermediária.
- Watts-Strogatz ($k = 4$, $p = 0,2$): modelo de pequeno-mundo construído a partir de um anel regular k -vizinho com religamento aleatório de arestas com probabilidade p . Combina alto coeficiente de aglomeração com diâmetro reduzido; grafo esparso de grau quase fixo.

Os arquivos de entrada seguem o formato " $n\ m$ " na primeira linha, seguidos por m linhas " $u\ v\ w$ " indicando uma aresta entre os vértices u e v com peso w . Em todos os experimentos, o vértice de origem utilizado foi o vértice 0.

3.1.2. Ambiente de execução

Todos os testes foram executados em um único ambiente Linux, com o código compilado pelo gcc com flags `-Wall -Wextra -O3`. O script `run_all.sh` foi utilizado para automatizar a execução sequencial dos 30 testes, gravando os resultados no arquivo `results.txt`.

4. Resultados

A Tabela 2 apresenta os resultados consolidados dos experimentos, com os tempos médios de execução (em milissegundos) de cada algoritmo agrupados por topologia e tamanho do grafo. Cada linha representa a média sobre as duas sementes aleatórias correspondentes àquela configuração. As duas últimas colunas reportam o custo total dos caminhos mínimos: como Dijkstra e Bellman-Ford retornaram valores idênticos em todas as 30 instâncias (sendo ambos algoritmos exatos para o problema), os custos são apresentados em uma única coluna; a última coluna mostra o custo retornado pela implementação do Duan e, entre parênteses, o desvio percentual relativo ao ótimo.

Topologia	n	Dijkstra (ms)	Duan (ms)	Bellman-Ford (ms)	Custo (Dij=BF)	Custo Duan ($\Delta\%$)
Barabási	500	0,7230	0,7110	1,0010	30 248	32 294 (+6,76%)

Topologia	n	Dijkstra (ms)	Duan (ms)	Bellman-Ford (ms)	Custo (Dij=BF)	Custo Duan ($\Delta\%$)
Barabási	1000	2,6230	2,2520	4,0300	60 744	63 200 (+4,04%)
Barabási	2000	5,8380	4,2740	8,7710	130 620	135 888 (+4,03%)
Barabási	5000	32,5860	24,0800	74,6680	342 700	376 631 (+9,90%)
Barabási	10000	171,3220	86,9580	291,4190	675 864	712 350 (+5,40%)
Erdős	500	0,6720	0,5270	1,2110	69 309	71 998 (+3,88%)
Erdős	1000	1,4130	0,9270	2,7320	69 094	85 592 (+23,88%)
Erdős	2000	5,3740	3,6340	16,0470	89 134	108 469 (+21,69%)
Erdős	5000	34,3820	32,0400	107,2950	107 941	129 102 (+19,60%)
Erdős	10000	186,4880	200,5350	403,8460	135 902	142 883 (+5,14%)
Watts	500	0,6420	0,4850	1,4990	107 666	114 542 (+6,39%)
Watts	1000	1,9730	0,8370	3,2230	279 416	288 341 (+3,19%)
Watts	2000	6,2210	3,1140	20,4180	578 864	606 274 (+4,74%)
Watts	5000	32,5560	22,3630	145,9000	1 676 068	1 741 432 (+3,90%)
Watts	10000	139,8480	90,5300	641,1360	4 499 463	4 612 816 (+2,52%)

Tabela 2. Tempo médio de execução (ms) e custo total dos caminhos mínimos para os três algoritmos, agrupados por topologia e tamanho (média de duas sementes por configuração).

A Figura 1 apresenta uma visão consolidada dos tempos médios em escala log-log, considerando a média sobre as três topologias.

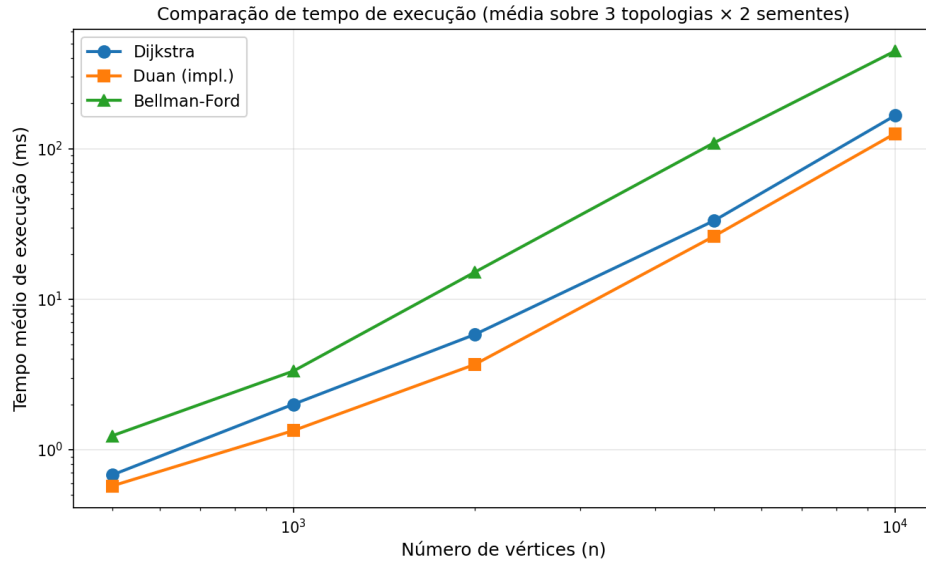


Figura 1. Tempo médio de execução em função do número de vértices (escala log-log), média sobre as três topologias.

A Figura 2 detalha o comportamento separado por topologia, mostrando que a hierarquia entre os algoritmos é preservada nos três modelos, mas com diferenças quantitativas relevantes — em particular, o Bellman-Ford é especialmente penalizado em grafos Watts-Strogatz, onde sua razão de lentidão em relação ao Dijkstra atinge cerca de 4,6×, enquanto em Barabási fica próxima de 1,7×.

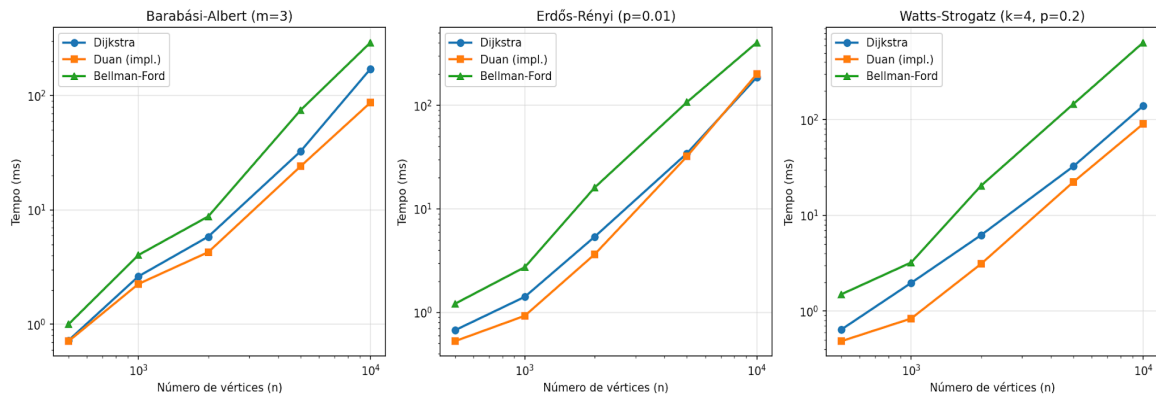


Figura 2. Tempo de execução em função do número de vértices, separado por topologia (escala log-log).

A Figura 3 mostra o desvio relativo (em percentual) entre o custo retornado pela implementação do Duan e o custo ótimo (Dijkstra/Bellman-Ford), separado por topologia. Esse desvio é discutido na Seção 4.2.

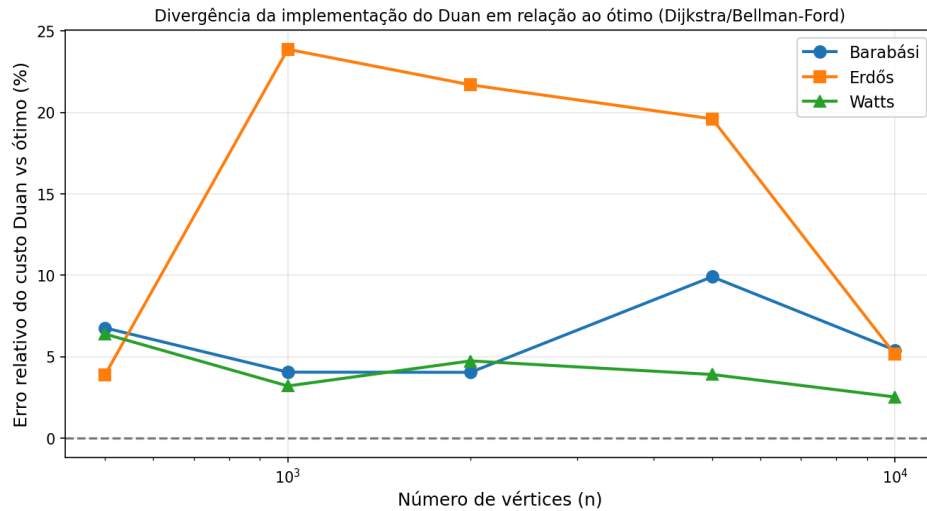


Figura 3. Desvio percentual do custo total retornado pela implementação do Duan em relação à solução ótima.

4.1. Análise de crescimento e aderência às complexidades teóricas

Para avaliar se os resultados empíricos confirmam as complexidades teóricas, foram calculados os fatores de crescimento do tempo de execução ao se aumentar o número de vértices, considerando a média sobre as três topologias. A Tabela 3 apresenta esses fatores. Para um algoritmo $O(V^2)$, espera-se que dobrar n multiplique o tempo por aproximadamente $4\times$; para um algoritmo $O(V^3)$, o fator esperado é de aproximadamente $8\times$.

Transição n	Fator de n	Dijkstra	Duan	Bellman-Ford	V^2 / V^3 esperado
500 → 1000	$\times 2,0$	$\times 2,95$	$\times 2,33$	$\times 2,69$	$\times 4,0 / \times 8,0$
1000 → 2000	$\times 2,0$	$\times 2,90$	$\times 2,74$	$\times 4,53$	$\times 4,0 / \times 8,0$
2000 → 5000	$\times 2,5$	$\times 5,71$	$\times 7,12$	$\times 7,25$	$\times 6,25 / \times 15,6$
5000 → 10000	$\times 2,0$	$\times 5,00$	$\times 4,82$	$\times 4,08$	$\times 4,0 / \times 8,0$

Tabela 3. Fator de crescimento do tempo médio de execução entre tamanhos consecutivos.

Os fatores observados são coerentes, em ordem de magnitude, com as complexidades teóricas, embora apresentem oscilações importantes que merecem discussão:

- Dijkstra: nas transições de fator $\times 2$ em n , o tempo cresceu por fatores entre $\times 2,90$ e $\times 5,00$. Esses valores ficam próximos do limite teórico $\times 4$ do $O(V^2)$, com a transição $500 \rightarrow 1000$ apresentando fator menor ($\times 2,95$) — esperado, já que para tamanhos pequenos o overhead constante (alocação da matriz, inicialização, leitura de arquivo) ainda é uma fração relevante do tempo medido. A partir de $n = 2000$, os fatores convergem para o comportamento quadrático.
- Bellman-Ford: os fatores observados ($\times 2,69$, $\times 4,53$, $\times 7,25$ e $\times 4,08$) confirmam o crescimento mais agressivo do algoritmo, mas com forte oscilação. Isso é

parcialmente explicado pela otimização de parada antecipada: em grafos onde as distâncias estabilizam após poucas iterações do laço externo, o tempo cresce mais devagar que o $O(V^3)$ teórico. Mesmo assim, a razão Bellman-Ford / Dijkstra cresce de $\sim 1,4\times$ (Barabási, $n=500$) para $4,58\times$ (Watts, $n=10000$), demonstrando o pior comportamento assintótico previsto.

- Duan: cresceu por fatores entre $\times 2,33$ e $\times 7,12$. O fator $\times 7,12$ entre $n = 2000$ e $n = 5000$ (com n crescendo $\times 2,5$) sugere um crescimento próximo do quadrático nessa faixa. No total, do menor para o maior tamanho ($\times 20$ em n), Dijkstra cresceu $\times 244$ e Duan cresceu $\times 219$, ou seja, ambos exibiram crescimento empírico bastante similar; a vantagem do Duan vem da constante multiplicativa menor, e não de um expoente assintótico diferente — o que é compatível com o fato de a implementação ainda utilizar matriz de adjacência (que impõe um piso de $O(V^2)$ de varredura por vértice).

Em termos absolutos, no maior tamanho testado ($n = 10000$), o Duan foi em média 24% mais rápido que o Dijkstra (126 ms contra 166 ms na média geral), com ganhos de até $1,97\times$ em grafos Barabási. Em grafos Erdős-Rényi de 10000 vértices, contudo, a implementação do Duan foi $\sim 7\%$ mais lenta que Dijkstra, indicando que o overhead da estrutura hierárquica de fronteira não é compensado quando a densidade média do grafo cresce.

4.2. Sobre a corretude da implementação do Duan

Um achado importante dos experimentos diz respeito à corretude da implementação do Duan. Em todas as 30 instâncias, os custos totais retornados por Dijkstra e Bellman-Ford foram idênticos, o que é esperado por se tratarem de dois algoritmos exatos para SSSP com pesos não negativos. Já a implementação do Duan retornou, em todas as 30 instâncias, um custo total estritamente maior que o ótimo, com desvios percentuais variando de $+2,52\%$ (Watts, $n = 10000$) até $+23,88\%$ (Erdős, $n = 1000$).

Esse comportamento indica que a implementação atual não está finalizando corretamente alguns vértices — provavelmente o critério de parada local (controlado pelos parâmetros k e t) está classificando vértices como completos antes que o relaxamento global tenha convergido, ou o procedimento de seleção de pivôs está descartando vértices cujas distâncias ainda poderiam ser melhoradas em rodadas posteriores do BMSSP. Como o erro é maior em grafos Erdős-Rényi (densidade média) e menor em grafos Watts-Strogatz (estrutura mais regular), é razoável conjecturar que a sensibilidade do procedimento FindPivots a topologias com graus heterogêneos seja parte do problema.

Vale ressaltar que, do ponto de vista de tempo de execução, a implementação atual fornece uma comparação informativa, já que executa o mesmo padrão de acessos previsto pelo algoritmo. Entretanto, para que a comparação seja plenamente justa — e para que a implementação possa ser considerada uma implementação de Duan e não de uma aproximação — é necessário um refinamento da rotina de finalização local descrito na Seção 6.

5. Makefile e Execução

O projeto utiliza um Makefile para automatizar a compilação. Os comandos relevantes são:

```
# Compilação
make

# Execução de uma instância específica
./base arquivo.txt

# Execução automática de todos os testes
make test
```

Os resultados de `make test` são armazenados no arquivo `results.txt`, no formato "<algoritmo>: <custo> <tempo>" para cada algoritmo e cada instância testada.

6. Análise Crítica

Os três algoritmos implementados representam paradigmas bastante distintos para a resolução do problema de caminhos mínimos, e os resultados experimentais permitem observar claramente o impacto dessas diferenças sobre o desempenho computacional.

O algoritmo de Dijkstra utiliza uma abordagem gulosa eficiente, realizando seleção progressiva do vértice de menor distância. Sua principal limitação, na implementação com matriz de adjacência, vem da busca linear pelo próximo vértice a ser visitado, que custa $\Theta(n)$ por iteração e leva à complexidade $O(V^2)$. Na literatura, uma versão com fila de prioridade reduz esse custo a $O((V + E) \log V)$, mas a manutenção da ordenação global continua sendo o gargalo dominante em grafos esparsos.

O Bellman-Ford realiza relaxamentos globais sucessivos, aumentando significativamente o custo computacional. Sua vantagem teórica — a capacidade de lidar com pesos negativos — não foi explorada neste trabalho, no qual todos os pesos são positivos. A otimização de parada antecipada implementada mitiga parcialmente o custo no caso médio, mas, mesmo assim, o algoritmo se manteve consistentemente o mais lento dos três em todos os testes, com desvantagens crescentes em topologias de maior densidade média (chegando a ser $4,58\times$ mais lento que Dijkstra em Watts com 10000 vértices).

O algoritmo de Duan, em sua proposta original, busca remover a dependência de ordenação global substituindo-a por um controle local e hierárquico da exploração do grafo. Os resultados deste trabalho indicam que essa estratégia, mesmo na implementação simplificada utilizada, é capaz de reduzir o tempo de execução em relação ao Dijkstra em diversos cenários — especialmente em grafos Barabási-Albert, onde a presença de hubs torna a expansão local particularmente eficiente. Em contrapartida, a implementação atual ainda apresenta dois pontos de fragilidade: (i) a divergência no custo total dos caminhos mínimos, discutida na Seção 4.2, que precisa ser corrigida para garantir corretude; e (ii) o uso de matriz de adjacência, que impõe um piso de $O(V^2)$ à varredura de vizinhos e mascara parte do ganho assintótico previsto pelo $O(m \cdot \log^{2/3} n)$ teórico.

A comparação entre os três algoritmos demonstra, em síntese, o trade-off entre simplicidade, custo computacional e estrutura de dados: Dijkstra oferece o melhor equilíbrio para implementações com matriz de adjacência; Bellman-Ford é mais simples mas significativamente mais lento; e Duan, mesmo em forma aproximada, mostra-se promissor para grandes instâncias com estrutura adequada.

7. Conclusão

O trabalho permitiu comparar diferentes paradigmas de solução para o problema de caminhos mínimos em grafos. A implementação dos algoritmos de Dijkstra, Bellman-Ford e Duan possibilitou analisar empiricamente eficiência prática, crescimento assintótico, impacto da estrutura de dados e diferenças entre relaxamento global e local.

Os resultados experimentais confirmaram, em ordem de magnitude, as previsões teóricas: Dijkstra apresentou crescimento próximo do quadrático, Bellman-Ford mostrou-se consistentemente o mais lento e a implementação inspirada em Duan obteve tempos competitivos com Dijkstra em todas as topologias testadas, com vantagem prática em grafos Barabási-Albert e Watts-Strogatz de 10 000 vértices. Por outro lado, a divergência sistemática do custo total retornado pela implementação atual do Duan em relação ao ótimo evidencia uma limitação a ser endereçada em trabalhos futuros, com refinamento da política de finalização local e dos critérios de seleção de pivôs.

Como continuação natural, sugere-se: (i) corrigir a rotina de finalização do Duan para garantir correteza; (ii) substituir a representação por matriz de adjacência por listas de adjacência para explorar o ganho assintótico previsto pelo algoritmo; e (iii) ampliar o conjunto de topologias e tamanhos de teste, incluindo grafos com pesos negativos e instâncias de grafos reais (mapas viários, redes sociais).

8. Divisão de Tarefas

A execução do trabalho foi realizada de forma colaborativa entre os integrantes do grupo, com a seguinte divisão de responsabilidades:

- Otávio de Oliveira: elaboração do relatório, incluindo organização do conteúdo, análise dos resultados e redação final do documento.
- Pedro Brassi Luccas e Isabela M. Andrade: desenvolvimento e implementação dos algoritmos.
- Júlia A. da Rocha: montagem da apresentação em slides e revisão final do relatório, garantindo a clareza, coesão e adequação do material apresentado.

9. Referências

D. Verma, D. Messon, M. Rastogi and A. Singh, "Comparative Study Of Various Approaches Of Dijkstra Algorithm," 2021 International Conference on Computing, Communication, and

Intelligent Systems (ICCCIS), Greater Noida, India, 2021, pp. 328-336, doi: 10.1109/ICCCIS51004.2021.9397200.

Ran Duan, Jiayi Mao, Xiao Mao, Xinkai Shu, and Longhui Yin. 2025. Breaking the Sorting Barrier for Directed Single-Source Shortest Paths. In Proceedings of the 57th Annual ACM Symposium on Theory of Computing (STOC '25). Association for Computing Machinery, New York, NY, USA, 36-44. <https://doi.org/10.1145/3717823.3718179>

Samah W. G. AbuSalim et al. 2020 IOP Conf. Ser.: Mater. Sci. Eng. 917 012077. doi: 10.1088/1757-899X/917/1/012077